

Recommended Revisions for the QCE Poster Paper

Email draft to Ibrahim

Subject: Recommended revisions for the QCE poster paper

Hi Ibrahim,

Based on Priya and Ulrike’s comments, I think the revised paper should be reorganized in a clearer “problem-definition first” style. The paper should first explain exactly what mathematical problem is being solved, then define the variables, constraints, cost functions, solver pipeline, and contribution. Only after that should we introduce the HUBO Hamiltonian.

I suggest revising the paper around the following structure.

1. Start with a clear discrete problem statement

We should first state that we are not directly solving the full continuous Model Predictive Control (MPC) problem from Masnavi et al. Instead, we are constructing a discrete grid-based benchmark inspired by that problem.

A possible wording is:

We consider a finite grid graph

$$G = (V, E), \tag{1}$$

where V is the set of grid cells and E is the set of valid one-step transitions. The UAV plans over discrete time steps

$$t = 0, 1, \dots, T. \tag{2}$$

The UAV starts at cell

$$s \in V \tag{3}$$

and should reach a target or terminal cell

$$g \in V. \tag{4}$$

The obstacle set is

$$O \subset V, \tag{5}$$

and the UAV is not allowed to occupy cells in O . At each time step, the UAV may move to a neighboring cell or remain in place. In addition to obstacle avoidance, the trajectory is evaluated by line-of-sight visibility to the target.

For each cell $u \in V$, let

$$N(u) = \{v \in V : (u, v) \in E\} \tag{6}$$

denote the set of cells reachable from u in one time step, including u itself if waiting is allowed.

Let

$$\text{LOS}(u, g) \tag{7}$$

denote the set of grid cells intersected by the discrete line segment from cell u to target cell g , for example using Bresenham's algorithm. A cell u is visibility-occluded if

$$\text{LOS}(u, g) \cap O \neq \emptyset. \quad (8)$$

Let B be the set of non-obstacle buffer cells adjacent to obstacles:

$$B = \{v \in V \setminus O : \exists o \in O \text{ such that } (v, o) \in E\}. \quad (9)$$

In words, B is the set of safe but risky cells near obstacles. These cells are not forbidden, but lingering in them is discouraged.

This will address the comments about the problem not being described satisfactorily, LOS not being defined, buffer cells not being defined, and B not being defined.

2. Define the decision variables before the Hamiltonian

Before writing the Hamiltonian H , we should define the binary variables clearly. The main decision variable is

$$x_{t,v} \in \{0, 1\}, \quad (10)$$

where

$$x_{t,v} = 1 \quad (11)$$

means that the UAV is at grid cell v at time step t , and

$$x_{t,v} = 0 \quad (12)$$

means that the UAV is not at grid cell v at time step t .

A binary solution is decoded into a trajectory

$$\pi = (v_0, v_1, \dots, v_T), \quad (13)$$

where v_t is the unique cell for which

$$x_{t,v_t} = 1. \quad (14)$$

This decoding is valid only if the one-hot occupancy constraint is satisfied.

3. Define the constraints before listing the Hamiltonian terms

The paper should clearly state the feasibility constraints in words before turning them into penalties.

The one-hot occupancy constraint is

$$\sum_{v \in V} x_{t,v} = 1, \quad t = 0, \dots, T. \quad (15)$$

This means that at each time step, the UAV occupies exactly one grid cell.

The start constraint is

$$x_{0,s} = 1. \quad (16)$$

This means that the UAV starts at the prescribed start cell s .

The motion constraint is

$$x_{t,u}x_{t+1,v} = 0, \quad v \notin N(u). \quad (17)$$

This means that the UAV cannot move from u to v if v is not reachable from u in one time step.

The obstacle-avoidance constraint is

$$x_{t,v} = 0, \quad v \in O. \quad (18)$$

This means that the UAV cannot occupy obstacle cells.

If the implementation treats the target as a hard terminal condition, then we should write

$$x_{T,g} = 1. \quad (19)$$

If the implementation treats target reaching as a soft cost, then the paper should say that reaching the target is encouraged through H_{term} instead of being enforced as a hard constraint.

This is important: we should not mix hard and soft terminal conditions unless the code actually does both.

4. Define each cost or penalty term clearly

After the constraints are stated, we can define the HUBO terms. The full Hamiltonian can be written as

$$H(x) = H_{\text{uniq}} + H_{\text{start}} + H_{\text{move}} + H_{\text{obs}} + H_{\text{occ}} + H_{\text{prox}} + H_{\text{term}}. \quad (20)$$

The one-hot occupancy penalty is

$$H_{\text{uniq}} = \lambda_{\text{uniq}} \sum_{t=0}^T \left(\sum_{v \in V} x_{t,v} - 1 \right)^2. \quad (21)$$

This term is zero only when the UAV occupies exactly one cell at each time step.

The start-cell penalty is

$$H_{\text{start}} = \lambda_{\text{start}}(1 - x_{0,s}). \quad (22)$$

The invalid-motion penalty is

$$H_{\text{move}} = \lambda_{\text{move}} \sum_{t=0}^{T-1} \sum_{u \in V} \sum_{v \notin N(u)} x_{t,u} x_{t+1,v}. \quad (23)$$

This prevents teleportation or invalid jumps.

The obstacle-avoidance penalty is

$$H_{\text{obs}} = \lambda_{\text{obs}} \sum_{t=0}^T \sum_{v \in O} x_{t,v}. \quad (24)$$

This should be treated as a hard-constraint penalty with a large weight.

For the occlusion or visibility cost, first define the per-cell occlusion cost

$$C_{\text{occ}}(u) = |\text{LOS}(u, g) \cap O|. \quad (25)$$

This is the number of obstacle cells blocking the line of sight from cell u to the target g . Then the occlusion term is

$$H_{\text{occ}} = \lambda_{\text{occ}} \sum_{t=0}^T \sum_{u \in V} C_{\text{occ}}(u) x_{t,u}. \quad (26)$$

In words, H_{occ} adds a visibility penalty whenever the UAV occupies a cell from which the target is occluded.

The sustained-proximity term is

$$H_{\text{prox}} = \lambda_{\text{prox}} \sum_{b \in B} \sum_{t=1}^{T-1} x_{t-1,b} x_{t,b} x_{t+1,b}. \quad (27)$$

This penalizes staying in the same buffer cell b for three consecutive time steps. This is the main cubic term and is the clearest reason the formulation is HUBO rather than QUBO.

If the terminal condition is soft, the terminal penalty can be written as

$$H_{\text{term}} = \lambda_{\text{term}}(1 - x_{T,g}). \quad (28)$$

If the terminal condition is hard, then $x_{T,g} = 1$ should be listed as a constraint and H_{term} should either be removed or explained differently. The paper should match the code exactly.

5. Explain hard constraints versus soft costs

This is important for defining feasibility-aware benchmarking. The hard-constraint penalties are

$$H_{\text{uniq}}, \quad H_{\text{start}}, \quad H_{\text{move}}, \quad H_{\text{obs}}. \quad (29)$$

The soft trajectory-quality terms are

$$H_{\text{occ}}, \quad H_{\text{prox}}, \quad H_{\text{term}}, \quad (30)$$

depending on how H_{term} is implemented.

Feasibility-aware benchmarking should be defined as follows:

Feasibility-aware benchmarking means that we do not judge a solver only by the total penalized energy $H(x)$. After each solver returns a binary sample, we decode it into a UAV trajectory and independently check whether the hard constraints are satisfied. Only then do we report soft cost, visibility, target reaching, and runtime.

This addresses the comments about “feasibility-aware,” “raw energy,” and “raw HUBO.” I suggest avoiding the phrase “raw HUBO.” Instead, use “total penalized energy” or “aggregate HUBO energy.”

6. Explain the solver pipeline clearly

The paper should be very explicit about which solver solves what.

A*: A* is a classical graph-search baseline. It does not solve the HUBO directly. It searches the grid using obstacle, occlusion, and proximity costs. Its role is to show that the discretized planning problem produces a sensible classical path.

Custom simulated annealing: Custom simulated annealing evaluates the original higher-order HUBO objective directly. This is the native-HUBO classical baseline.

neal: neal is not a quantum solver and not a direct HUBO solver. It is a classical simulated-annealing sampler for BQM/QUBO models. Therefore, the correct pipeline is

$$\text{HUBO} \longrightarrow \text{quadratzation} \longrightarrow \text{QUBO/BQM} \longrightarrow \text{neal}. \quad (31)$$

Then neal samples the quadratized QUBO/BQM. The returned sample must be decoded back into a UAV trajectory and checked independently for feasibility.

If the code uses `dimod.HigherOrderComposite`, we should say:

The HUBO is passed through `dimod.HigherOrderComposite`, which internally quadratizes the higher-order model before calling neal.

If the code uses `dimod.make_quadratic`, we should say:

The HUBO is quadratized using `dimod.make_quadratic` before being sampled by neal.

We need to verify which one is actually used in the code.

Qiskit QAOA: Qiskit QAOA should be described carefully. A safe statement is:

Qiskit QAOA was implemented only on the reduced 2×2 , 8-qubit QUBO instance. It was not applied to the full 960-variable HUBO. It is included as a reduced-instance correctness and compatibility check, not as a scalable solution to the full problem.

This directly answers the question: yes, Qiskit is implemented, but only for the tiny reduced QUBO case. We should not say that QAOA solves the full HUBO. We should also avoid the phrase “QAOA-style methods” unless it is clearly defined.

7. Verify quadratzation statistics from the code

Before resubmission, we should verify all numerical claims directly from the current code:

- grid size,
- planning horizon T ,

- number of original binary variables,
- number of cubic terms,
- number of auxiliary variables after quadratization,
- final QUBO/BQM variable count,
- final QUBO/BQM interaction count,
- whether `neal` uses `HigherOrderComposite` or `make_quadratic`,
- whether Qiskit QAOA is run only on the reduced QUBO,
- whether the terminal condition is hard or soft.

The paper should report only numbers that are reproducible from the current implementation. This is especially important because the quadratization/ancilla count is a point that Priya and Ulrike may check carefully.

8. State the contribution conservatively

The contribution should not be framed as quantum speedup or as a full quantum solution. A stronger and safer contribution statement is:

The contribution is not a quantum speedup claim. The paper introduces a discrete HUBO benchmark for obstacle-avoidance and occlusion-aware UAV trajectory planning, clarifies the cost of quadratizing this higher-order model for QUBO/BQM solvers, and proposes feasibility-aware benchmarking to prevent low-energy but infeasible samples from being misreported as good solutions.

This is defensible and directly addresses the reviewers' concerns.

9. Suggested revised paper structure

I recommend this structure:

- I. **Introduction and contribution.** Briefly explain the Masnavi continuous MPC problem, state that we build a discrete HUBO benchmark, and clearly say there is no quantum speedup claim.
- II. **Discrete problem definition.** Define G , V , E , T , s , g , O , B , $N(u)$, $\text{LOS}(u, g)$, and $x_{t,v}$. Explain trajectory decoding.
- III. **Constraints, costs, and HUBO formulation.** First list the feasibility constraints and soft costs, then assemble the Hamiltonian $H(x)$.

- IV. **Solver pipeline and feasibility-aware benchmarking.** Explain A*, custom simulated annealing, neal after quadratization, Qiskit QAOA only on reduced QUBO, and independent feasibility checking.
- V. **Results and discussion.** Report the solver table, quadratization statistics, penalty-weight sweep, and QAOA reduced-instance result.
- VI. **Conclusion.** State contribution and limitations: moving target, real D-Wave hardware, comparison with continuous MPC, per-step visibility logging, and code release.

10. Main message

The revised paper should not start from the Hamiltonian. It should start from the problem. The key order should be

Problem $\longrightarrow$ variables $\longrightarrow$ constraints $\longrightarrow$ costs $\longrightarrow$ HUBO $\longrightarrow$ quadratization $\longrightarrow$ solvers $\longrightarrow$ feasibility check –
(32)

This ordering should address Ulrike’s main concern that the problem, variables, constraints, cost functions, and solver pipeline were not sufficiently clear.

Best,
Ali